

CONNECTING THE DOTS

*A Knowledge Graph for Exposing Fraud-Linked
Transaction Patterns in Banking Data*

Victoria Bassey · 948253

Data Semantics DS2526 · May 2026

[GitHub](#)

5M+

TRANSACTIONS

5,189

FRAUD CASES

\$187B

LAUNDERED

243×

TOP SUSPECT



What This Presentation Covers

1. The Problem: why flat data hides fraud
2. Idea, Objectives & Tools
3. The Complete Pipeline
4. Dataset Description
5. Data Quality Checks
6. Stratified Sampling
7. Ontology Design & Class Hierarchy

8. Building the Knowledge Graph
9. Fraud Patterns & SPARQL Queries
10. Findings & Demonstration
11. Full-Dataset Scaling & Deep Findings
12. Evaluation & Limitations
13. Conclusion & Future Work
14. References & Responsible AI

Why Flat Data Hides Fraud

The Flat CSV Problem

Fraudulent transactions are built to look like noise. On their own, they are isolated, normal looking rows in a database. A single transaction tells you nothing. A network of linked transactions tells the whole story.

The problem is not the data. It is the shape of the data.

A flat table forces investigators to read row by row. A knowledge graph lets them see the network. Fraud lives in the network, not in the rows.

THE SOLUTION



Raw CSV (5M rows)



Python → RDF Triples



GraphDB Triplestore



SPARQL Queries



Fraud Patterns Exposed

CORE ARGUMENT: Flat CSV represents rows. Knowledge graphs represent relationships. Semantic structures expose what isolated records conceal.

What We Set Out to Do and Why

The Core Idea

Instead of reading banking transactions as rows in a table, represent them as connected entities in a graph: banks, accounts and transactions, linked by the relationships between them. Once the links are explicit, money flows you cannot see in a table become easy to follow.

The graph is built in two tiers: a 12-transaction micro-prototype where every FanOut, Cycle and GatherScatter pattern is hand-verifiable, then a 10,177-transaction stratified subset that confirms the same structures hold at realistic scale.

Our Objectives

- Model banks, accounts, transactions and fraud indicators in an ontology.
- Convert transaction records into RDF.
- Store and query the graph in GraphDB.
- Show how semantic links expose fraud patterns hidden in tables.

Tools and Why

Protégé

Chosen over hand writing OWL because it makes classes and properties visual and catches modelling mistakes early.

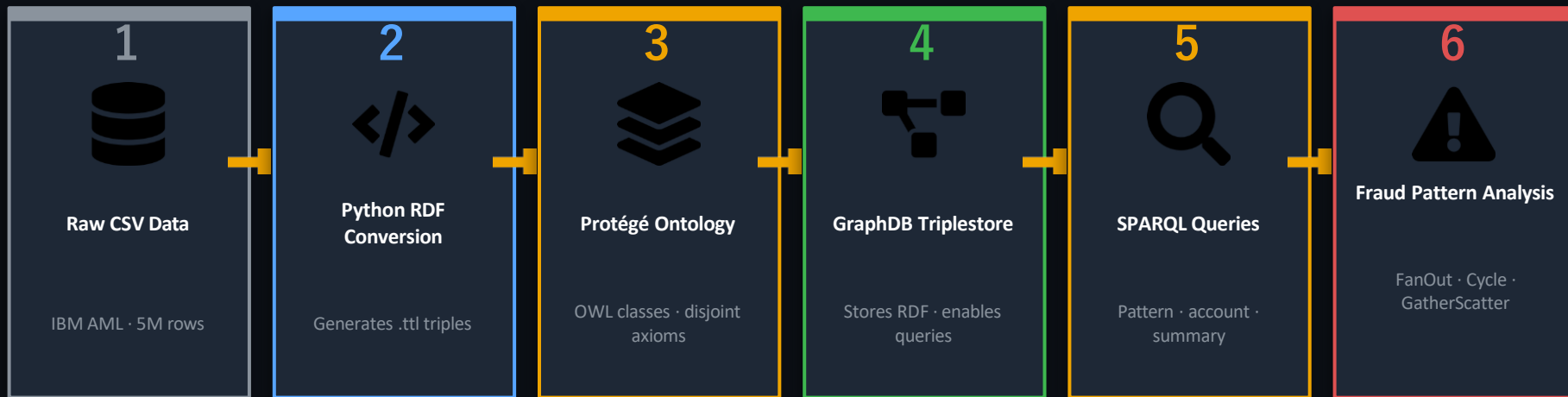
GraphDB

Chosen over a plain triple file because it stores RDF natively and runs SPARQL directly, which other stores would need extra setup for.

Python

Chosen to automate the CSV to RDF conversion rather than build triples by hand.

The Complete Pipeline



Raw → Fragmented

Flat CSV: disconnected rows, no relationships visible

Semantic → Connected

RDF triples link every entity: accounts, banks, transactions

Query → Explainable

SPARQL traverses the linked graph structure — every result is transparent and traceable back to the ontology

IBM Anti-Money Laundering Dataset

Synthetic dataset by IBM Research (Kaggle). Designed for AML research with explicit laundering labels. Covers 7 payment formats across real-world transaction patterns.

5,078,345

Total Transactions

5,177

Labelled Laundering

10,189

In KG

7

Payment Formats

♦ Stratified sampling: All 5,177 laundering cases retained + 5,000 random normal transactions. No fraud signal excluded.

Payment Format Distribution



DATA QUALITY CHECK

Missing Values

0

Negative Amounts

0

Duplicates Removed

9

Max Single Transaction

\$1T+

The \$1T+ max is a synthetic stress outlier in the raw IBM file, a normal-labelled row, not part of the \$187B laundering total (largest laundering transfer: \$84.8B). We retained it rather than capping it, to confirm the graph parses extreme scale without schema breakdown.

IBM Anti-Money Laundering Dataset

What we did. We used the IBM Anti-Money Laundering dataset from Kaggle, a synthetic dataset built for AML research. Every row is labelled as laundering or normal, which lets us check our results against ground truth.

Attributes used, and what each becomes in the graph

Timestamp	when the transfer happened (Transaction property)
From Bank / To Bank	the two institutions (Bank entities)
Sender / Receiver Account	the accounts moving money (Account entities)
Amount Paid / Received	value sent and received (Transaction property)
Payment / Receiving Currency	currencies involved (Currency entity)
Payment Format	method such as ACH or Wire (PaymentFormat entity)
Is Laundering	fraud flag (FraudIndicator)

Why these attributes

Together they give us everything a transaction needs in the graph: who sent money, who received it, through which bank, in what currency, by what method, and whether it was flagged. That is exactly the set of entities and relationships the knowledge graph is built from.

Checking the Data Before We Trust It

What we did. Before building anything, our Python script ran quality checks on the raw file, because any error in the data would carry straight through into the graph and into our conclusions.

Checks performed

- Missing values
- Duplicate records
- Negative or invalid amounts
- Data type consistency
- Currency code validity

Actions taken

- Dropped incomplete records
- Removed 9 duplicate transactions
- Removed invalid amounts
- Standardised text fields
- Confirmed every currency code

Result of the check

Missing values	0
Negative amounts	0
Duplicates removed	9

Outcome: under 0.1% of rows removed. The data was clean enough to trust for graph construction.

Why We Sampled, and How

Why. The full dataset holds over 5 million transactions, too many for a prototype graph. But fraud is rare, so a plain random sample would throw most of it away. Stratified sampling solves both problems: keep all the fraud, sample the rest.

Keep 100% of fraud

All 5,177 labelled laundering transactions were kept.
No fraud signal was thrown away.

Sample the normal

5,000 non-fraud transactions were drawn at random to give the graph a realistic background of ordinary activity.

Outcome: a balanced working set of 10,177 transactions that keeps every fraud case while staying small enough to build and query quickly.

Modelling Banking Entities and Relationships as Semantic Structures

OWL Class Hierarchy

```

owl:Thing
├─ Entity (superclass)
│   ├── Bank
│   ├── Account
│   └─ Transaction
│       └─ FraudulentTransaction
├─ PaymentFormat
├─ Currency
├─ FraudIndicator
└─ LaunderingPattern
  
```

Sibling Classes are pairwise disjoint.

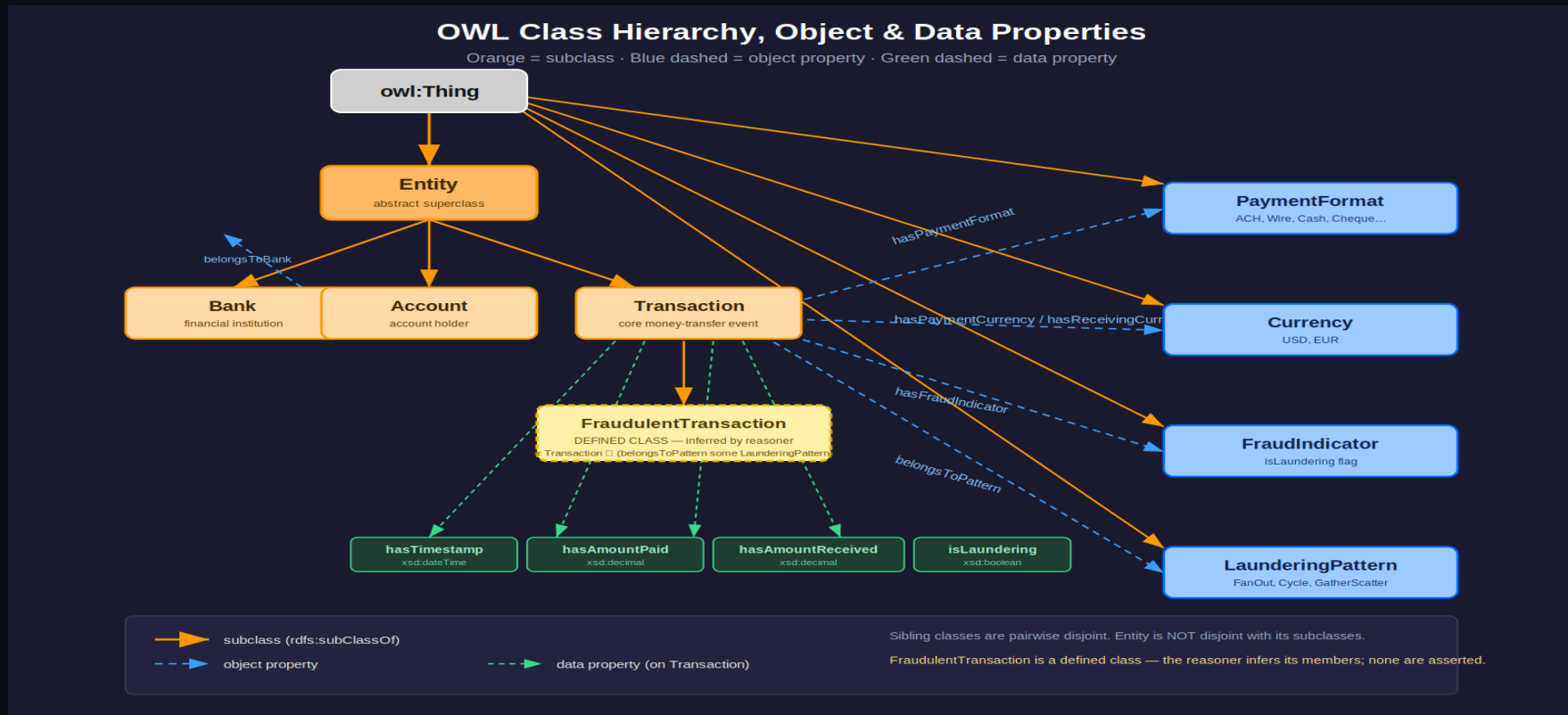
Object Properties

hasSenderAccount	hasFraudIndicator	hasPaymentFormat
Domain: Transaction	Domain: Transaction	Domain: Transaction
Range: Account	Range: FraudIndicator	Range: PaymentFormat
hasReceiverAccount	belongsToPattern	hasReceivingCurrency
Domain: Transaction	Domain: Transaction	Domain: Transaction
Range: Account	Range: LaunderingPattern	Range: Currency
belongsToBank	hasPaymentCurrency	
Domain: Account	Domain: Transaction	
Range: Bank	Range: Currency	

Domain/Range constraints prevent invalid triples

Semantic Validation: HermiT reasoner verifies consistency, disjointness, and domain/range constraints. Result: Zero ontological inconsistencies.

Class Hierarchy, Object & Data Properties



hasTimestamp is xsd:dateTime — ISO 8601 format (2022-09-01T00:06:00) — enabling temporal reasoning

How the Knowledge Graph Was Built

METHODOLOGY

1 Raw CSV Data

IBM AML · 5M rows



2 Filter & Sample

5,177 fraud + 5,000 normal transactions

3 Python to RDF

Convert rows to triples

4 OWL Ontology

Schema + HermiT valid

5 From GraphDB Load to OWL 2 RL

Triplestore, SPARQL ready

6 Query & Retrieve

FanOut · Cycle · GatherScatter

P S E U D O C O D E

INPUT: HI_Small_Trans.csv (5,078,345 rows)

OUTPUT: banking_fraud_kg_full.ttl

Step 1. Load CSV; remove negatives and duplicates

Step 2. Partition data:

fraud_rows = ls_Laundering = 1 [5,177 rows]

normal_rows = random sample [5,000 rows]

working_set = both combined [10,177 rows]

Step 3. For each row *r* in working_set:

create Transaction node (unique URI)

connect hasSenderAccount -> Account node

connect hasReceiverAccount -> Account node

record amount, timestamp, format, currency

record isLaundering = true or false

if fraud: assign pattern type

(FanOut / Cycle / GatherScatter)

connect Account -> belongsToBank -> Bank node

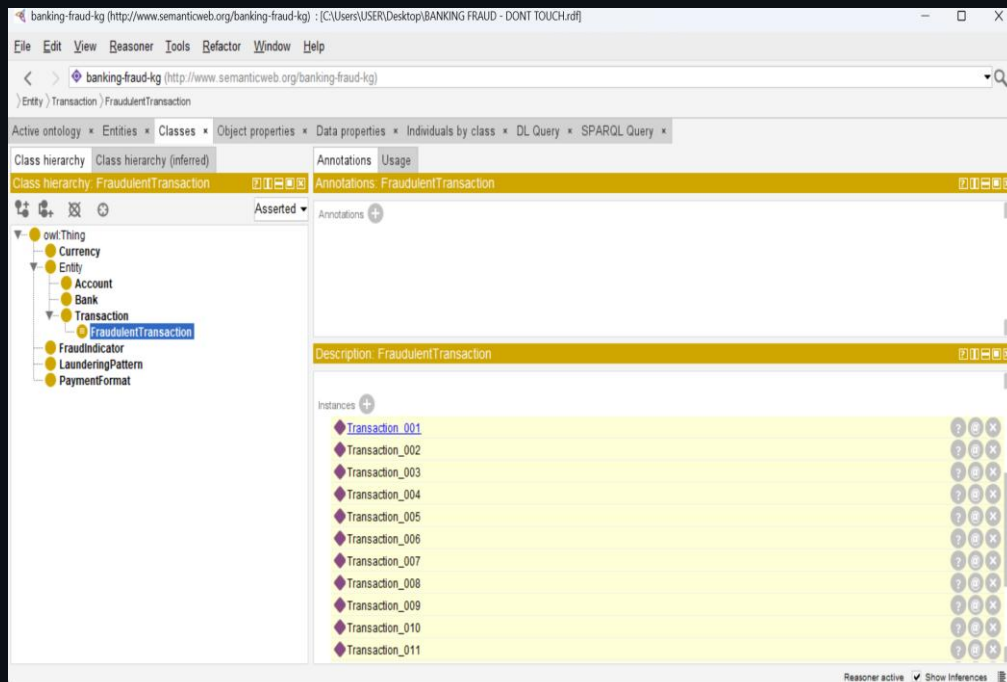
Step 4. Save as Turtle (.ttl) file

Step 5. Load OWL schema + .ttl into GraphDB

Step 6. Run HermiT reasoner -> STABLE

Semantic Inference in Action

The ontology derives new knowledge — it is not just storage



The defined class

FraudulentTransaction $\equiv$
 Transaction $\sqcap$ (belongsToPattern some LaunderingPattern)

No FraudulentTransaction instances were assigned by hand. When HerMiT ran, all 12 prototype transactions were automatically classified as members (shown yellow in Protégé).

This is logical deduction over the ontology's constraints — a database returns what you stored; a knowledge graph derives what logically follows.

KEY INSIGHT: classification is computed by the reasoner, not coded in Python — the semantics do the work.

How Transactions Become RDF Triples

Each transaction becomes a set of RDF triples (Subject - Predicate - Object):

txn:Transaction_001	rdf:type	Transaction
txn:Transaction_001	hasSenderAccount	acct:Account_800737690
txn:Transaction_001	hasReceiverAccount	acct:Account_80011F990
txn:Transaction_001	hasAmountPaid	"2848.96"^^xsd:decimal
txn:Transaction_001	hasTimestamp	"2022-09-01T00:06:00Z"^^xsd:dateTime
txn:Transaction_001	hasPaymentFormat	pf:ACH
acct:Account_800737690	rdf:type	Account
acct:Account_800737690	belongsToBank	bank:_021174 Bank

Why this matters semantically:

- Each entity has a unique URI (not just a cell in a table).
- Every relationship is explicit and named (hasSenderAccount, belongsToBank).
- The same data that sat in flat CSV rows now forms a connected graph.
- Because relationships are explicit, SPARQL can traverse them, a CSV cannot.

Navigating the RDF Graph Through Semantic Relationships

Example: Find all transactions with their connected entities

```
SELECT ?transaction ?sender ?receiver ?amount WHERE {  
    ?transaction    rdf:type                Transaction .  
    ?transaction    hasSenderAccount       ?sender .  
    ?transaction    hasReceiverAccount     ?receiver .  
    ?transaction    hasAmountPaid          ?amount .  
}
```

What makes this semantic, not just a query:

- SPARQL follows the RDF graph structure, it does not join flat tables.
- It understands that hasSenderAccount means "has as sender".
- It traverses named relationships between entities- Results come back as linked entities (transaction, sender, receiver), not isolated rows.
- This is why the same question is hard in SQL but natural in SPARQL.

The following SPARQL queries serve as competency questions used to evaluate whether the ontology can answer the questions it was designed for.

How Money Launderers Hide the Trail

FAN OUT



One account sends money to several different accounts at the same time — splitting the funds to make each transfer look unrelated.

WHAT HAPPENED IN THE DATA

Acct_800737690	→	Acct_80011F990	\$2,848
Acct_800737690	→	Acct_80020C5B0	\$4,200
Acct_800737690	→	Acct_8000EBD30	\$5,100

Same sender · 3 destinations · within 24 hrs

CYCLE



Money moves through a chain of accounts and ends up back where it started — making it nearly impossible to trace where the funds originally came from.

WHAT HAPPENED IN THE DATA

Acct_80020C5B0	→	Acct_800737690	\$8,000
Acct_800737690	→	Acct_8000EBD30	\$7,900
Acct_80098A3D0	→	Acct_800737690	\$9,100

Amounts vary each time to avoid triggering alerts

GATHER SCATTER



Money is collected from many different accounts into one, then sent back out to many others — so no single payer can be linked to any single receiver.

WHAT HAPPENED IN THE DATA

Acct_800EE6B70	→	FUNNEL	\$7,600
Acct_8000EBD30	→	FUNNEL	\$6,400
FUNNEL →	Acct_80098A3D0		\$6,900
Acct_800EE6B70	→	FUNNEL	\$18,721
FUNNEL →	Acct_80098A3D0		\$18,030

`Funnel = Acct_80044A5D0`

Q1: Full Network - All 12 Transactions at a Glance

K E Y I N S I G H T

This table shows all 12 transactions in the prototype graph, each linked to its sender and receiver account.

Notice Account_800737690 appears as sender three times in rows 1, 4 and 5. That is the FanOut pattern.

In a flat CSV these look like three separate, unrelated rows. In the graph the connection is immediately visible.

Rows 8, 9, 12 form the Cycle.

Rows 2, 3, 6, 7, 11 are GatherScatter. All three patterns surface at once from a single query.

P S E U D O C O D E

```
SELECT transaction, sender, receiver,
       pattern, format, amount, timestamp
```

For each Transaction node T in the graph:

```
find hasSenderAccount -> sender
find hasReceiverAccount -> receiver
find belongsToPattern -> pattern
find hasPaymentFormat -> format
find hasAmountPaid -> amount
find hasTimestamp -> timestamp
```

```
Return all rows ordered by transaction ID
No filter -> full network in one query
```

Network — All 12 Transactions at a Glance

GRAPHDB RESULT

Transaction	Sender Account	Receiver Account
Transaction_001	Account_800737690	Account_80011F990
Transaction_002	Account_800EE6B70	Account_80044A5D0
Transaction_003	Account_80044A5D0	Account_80098A3D0
Transaction_004	Account_800737690	Account_80020C5B0
Transaction_005	Account_800737690	Account_8000EBD30
Transaction_006	Account_800EE6B70	Account_80044A5D0
Transaction_007	Account_80044A5D0	Account_80098A3D0
Transaction_008	Account_80020C5B0	Account_800737690
Transaction_009	Account_800737690	Account_8000EBD30
Transaction_010	Account_80011F990	Account_80020C5B0
Transaction_011	Account_8000EBD30	Account_80044A5D0
Transaction_012	Account_80098A3D0	Account_800737690

Q2: FanOut

One Sender, Multiple Receivers

What this query does

Filters only Pattern_FanOut transactions then groups by sender to reveal accounts dispersing money in multiple directions.

No manual search needed — the query surfaces the suspect automatically.

One sender + multiple receivers

P S E U D O C O D E

```
SELECT sender, count(transactions),  
       receiver, amount, timestamp
```

```
Filter: pattern = Pattern_FanOut only
```

```
For each matching Transaction T:
```

```
find hasSenderAccount -> sender
```

```
find hasReceiverAccount -> receiver
```

```
find hasAmountPaid -> amount
```

```
find hasTimestamp -> timestamp
```

```
Group by sender -> count receivers each
```

```
Order by sender -> most active first
```

Q2: FanOut - One Sender, Multiple Receivers

KEY INSIGHT

The table confirms Account_800737690 sent money to three different accounts within a 24-hour window.

Row 2: \$2,848.96 on September 1.

Row 3: \$4,200.00 on September 2.

Row 4: \$5,100.00 on September 2.

One source, three destinations.
This is how launderers spread money to make each transfer look unrelated.

The diagram below shows Account_800737690 at the centre, with arrows pointing outward to each of the three receiver accounts.

GRAPHDB RESULT

Sender	#	Receiver	Amount	Timestamp
Acct_80011F990	1	Acct_80020C5B0	\$3,200.00	2022-09-05 10:12
Acct_800737690	1	Acct_80011F990	\$2,848.96	2022-09-01 00:06
Acct_800737690	1	Acct_80020C5B0	\$4,200.00	2022-09-02 09:15
Acct_800737690	1	Acct_8000EBD30	\$5,100.00	2022-09-02 11:20



Three sends inside 24 hours, amounts deliberately randomized — \$2,848 · \$4,200 · \$5,100 — to defeat static amount-matching. One sender, three receivers, one linked entity.

Q3: Cycle - Money Looping Through Accounts

What this query does

Retrieves all Pattern_Cycle transactions ordered by timestamp to expose circular flow of funds.

Money leaves an account, passes through intermediaries, and returns to origin.

Ordering by time reveals the sequence

P S E U D O C O D E

```
SELECT  transaction, sender, receiver,  
        amount, timestamp
```

```
Filter:  pattern = Pattern_Cycle  only
```

```
For each matching Transaction T:
```

```
find  hasSenderAccount  ->  sender
```

```
find  hasReceiverAccount ->  receiver
```

```
find  hasAmountPaid     ->  amount
```

```
find  hasTimestamp      ->  timestamp
```

```
Order by timestamp  ->  reveals the loop
```

```
Cycle: same account = sender AND receiver
```

Q3: Cycle - Money Looping Through Accounts

KEY INSIGHT

The table shows three transactions ordered by time, forming a loop.

Step 1: Acct_80020C5B0 sends \$8,000 to Acct_800737690 at 09:25.

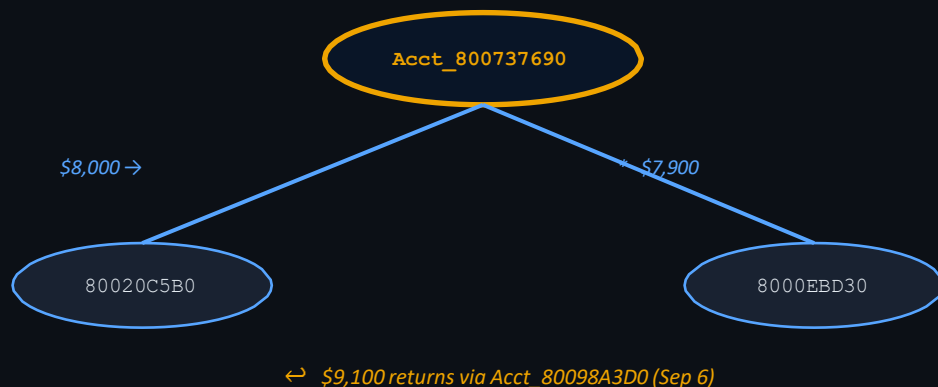
Step 2: Acct_800737690 sends \$7,900 to Acct_8000EBD30 at 13:40 (4 hrs later).

Step 3: Acct_80098A3D0 sends \$9,100 back to Acct_800737690 on Sep 6.

The money comes back to where it started. The amounts change each time on purpose to avoid triggering bank alert systems that flag identical repeated amounts.

GRAPHDB RESULT

Transaction	Sender	Receiver	Amount	Timestamp
Txn_008	Acct_80020C5B0	Acct_800737690	\$8,000.00	2022-09-04 09:25
Txn_009	Acct_800737690	Acct_8000EBD30	\$7,900.00	2022-09-04 13:40
Txn_012	Acct_80098A3D0	Acct_800737690	\$9,100.00	2022-09-06 11:32



● Q4: GatherScatter — Funnel Account Identification

GatherScatter funnels gather from many senders and scatter to many receivers. We identify them structurally — as accounts with both a high incoming AND a high outgoing count of fraudulent transactions — ranked by incoming volume across all 5,177 fraud transactions.

PSEUDOCODE

```
SELECT  ?funnel
        COUNT(?inTxn) AS ?incoming
        COUNT(?outTxn) AS ?outgoing

?inTxn  a bfg:FraudulentTransaction ;
        hasReceiverAccount ?funnel .
?outTxn a bfg:FraudulentTransaction ;
        hasSenderAccount   ?funnel .

GROUP BY ?funnel
ORDER BY DESC(?incoming)

Funnel = high IN AND high OUT
```

GRAPHDB RESULT

Funnel Account	In	Out
Account_811C599A0	31	21
Account_8021353D0	28	21
Account_811C597B0	23	21
Account_80266F880	22	29
Account_811ED7DF0	20	2

A funnel both receives and re-sends fraudulent funds — high incoming AND outgoing counts are its signature. Account_811C599A0 takes in 31 and sends out 21, passing money through largely unchanged to break the audit trail. The 20-in / 2-out row is a near-pure collector, where the pattern thins.

● Q5: Fraud Leaderboard — Most-Connected Accounts

Ranks every account by how many laundering transactions it appears in as sender or receiver across all 5,177 fraud transactions. Frequency, not transaction size, exposes the accounts most deeply embedded in the network: the persistent, repeat actors.

PSEUDOCODE

```
SELECT  ?account
        (COUNT(?txn) AS ?appearances)
```

```
?txn a bfgk:FraudulentTransaction .
{ ?txn hasSenderAccount  ?account }
UNION
{ ?txn hasReceiverAccount ?account }
```

```
GROUP BY  ?account
ORDER BY  DESC(?appearances)
```

243 hits = serial laundering hub

GRAPHDB RESULT

Account	Sent	Recv	Total
Account_100428660	243	0	243
Account_1004286A8	158	0	158
Account_811C599A0	21	31	52
Account_80266F880	29	22	51
Account_8021353D0	21	28	49
Account_811C597B0	21	23	44

Account_100428660 appears 243 times — always as sender — a serial dispersal hub steadily pushing dirty money out. Mid-table accounts (811C599A0: 31 in / 21 out) are balanced funnels woven into the network. The graph surfaces repeat offenders, not one-off large transfers.

Q6: Pattern Summary — From Prototype to Full Scale

The same structures also appear within the wider 5,177 fraud subset, demonstrating that FanOut, Cycle, and GatherScatter are consistent fraud signatures across the full dataset.

PROTOTYPE

12 transactions · detailed illustration

FanOut	4 txns	\$15,348.96
Cycle	3 txns	\$25,000.00
GatherScatter	5 txns	\$57,652.03
Total	12 txns	\$98,000.99

FULL DATASET — 5,177 FRAUD TXNS

accounts detected by structure, not by label

FanOut	130 accounts
Cycle · pass-through	1,008 accounts
GatherScatter funnels	80 accounts
Total laundered	\$187 B

130 dispersal hubs, 1,008 circular-flow accounts, 80 funnels identified across 5,177 fraud transactions by analyzing network structure. Accounts may overlap between categories.

12 Transactions. One Suspect. Three Fraud Stories.

FAN OUT

Acct_800737690 → **\$2,848.96**
Acct_80011F990

Acct_800737690 → **\$4,200.00**
Acct_80020C5B0

Acct_800737690 → **\$5,100.00**
Acct_8000EBD30

All sent within a 24-hour window. One source, three destinations.

Semantically: one entity linked outward to three separate receiver nodes.



Account_800737690

PRIMARY SUSPECT

CYCLE

Acct_80020C5B0 to **\$8,000.00**
Acct_800737690

Acct_800737690 to **\$7,900.00**
Acct_8000EBD30

Acct_80098A3D0 to **\$9,100.00**
Acct_800737690

Money returns to origin. Amounts vary each leg to avoid triggering rule-based alert thresholds.

Semantically: a directed cycle — each account is both sender and receiver.

6×

appearances in
12 transactions

\$98K

suspicious money
across 12 transactions

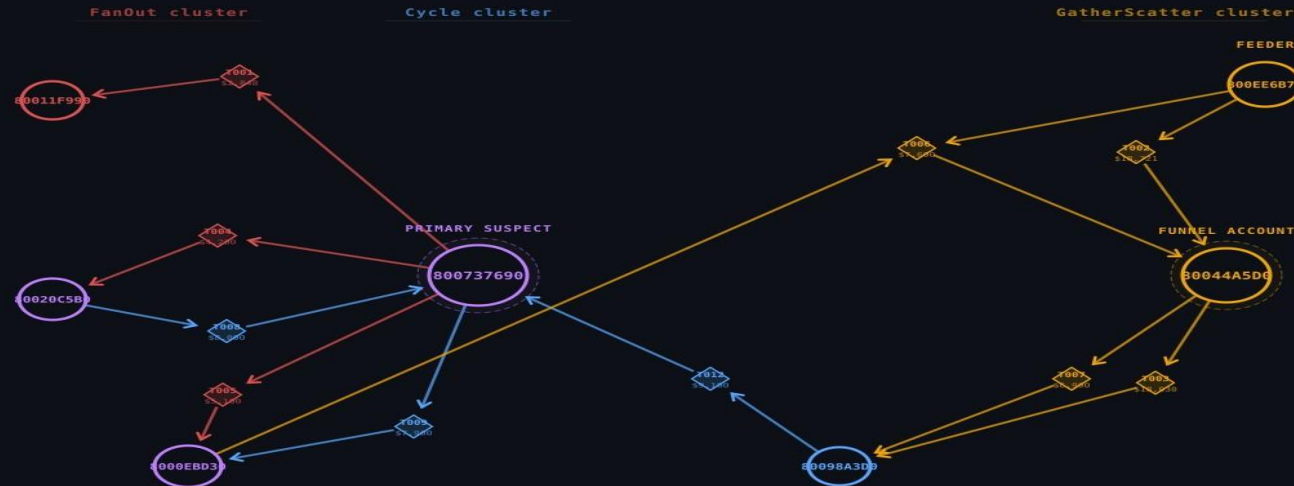
BOTH

FanOut + Cycle
patterns active



Confirmed across
full 5,177 dataset

The 12 Transaction Prototype Network



LEGEND

- FanOut — one sender disperses funds to multiple receivers
- Cycle — money loops through accounts back to origin
- GatherScatter — funnel collects from many, redistributes to many
- Hub — account linked across multiple laundering patterns simultaneously

HOW TO READ THIS GRAPH

Circles = bank accounts · Diamonds = transactions · Arrows = direction money flows · Node SIZE = number of appearances (bigger = more suspicious) · COLOUR = laundering pattern: red FanOut · blue Cycle · gold GatherScatter

Account_800737690 (purple, centre) — present in BOTH FanOut and Cycle simultaneously. One account at the heart of two overlapping fraud patterns. Account_80044A5D0 (gold, right) — the funnel: collects money from multiple sources and immediately redistributes it to break the audit trail.

A Pure FanOut Hub — 243 Sends, Nothing Back.

243 transactions. 236 accounts. \$50 million. 18 days. Each send small enough to look normal. Each receiver different enough to break the trail. This is industrial-scale dispersal masquerading as routine activity.



Scaling to 5,177 Laundering Cases

FRAUD LEADERBOARD — TOP 5 ACCOUNTS

#1 Account_100428660  243

#2 Account_1004286A8  153

#3 Account_811C599A0  52

#4 Account_80266F880  51

#5 Account_8021353D0  49

Account_100428660 — 243 appearances across confirmed laundering transactions, surfaced via a single SPARQL query traversing the linked graph. The same analysis on flat CSV would require sequential row-by-row scanning.

LAUNDERED BY PAYMENT FORMAT

Payment Format	Transactions	Total Laundered	Share
ACH	4,491	\$186.6B	87%
Cheque	324	\$178.5M	6%
Cash	108	\$255.4M	<2%
Credit Card	206	\$5.0M	<4%
Wire	4	\$31.5K	<1%
Bitcoin	56	\$35.02	<1%

87
%

of all laundering used ACH. Fast, widely available, harder to trace than cash. \$186 BILLION through one payment channel.

Mega-Transactions

The largest single laundering transfers in the dataset — each one a single row among five million, surfaced instantly by graph traversal.

Transaction_000236

\$84.8 B

LARGEST SINGLE TRANSFER
Credit Card · 2022-09-01

Transaction_000442

\$66.1 B

ACH · 2022-09-02

Transaction_002598

\$21.5 B

ACH · 2022-09-03

Transaction_003025

\$2.6 B

Cheque · 2022-09-04

Transaction_001512

\$2.5 B

ACH · 2022-09-05

Cross-Bank Smurfing in “Normal” Data

FLAG 1 · Institutional blind spot. Two normal-labelled hops, three different banks, under 60 minutes. The same \$176.74 is fed repeatedly into hub 100428660 — our #1 fraud account — then cleared straight back out across the bank perimeter.

PSEUDOCODE

```
SELECT  A, $in, B, $out, C, gap
WHERE {
  A belongsToBank BankA ;
  B belongsToBank BankB ;
  C belongsToBank BankC .
  Tx1 hasSender A ; receiver B ;
    hasAmountPaid $in ;
    isLaundering false .
  Tx2 hasSender B ; receiver C ;
    hasAmountPaid $out ;
    isLaundering false .
  FILTER(BankA != BankB != BankC)
  FILTER(0 <= gap <= 60 min)
}
ORDER BY gap
```

GRAPHDB RESULT

A-ACCT	IN \$	HUB (B)	GAP
80EE7A540	176.74	100428660	2m
80EE7A540	176.74	100428660	3m
80EE7A540	176.74	100428660	22m
801486E40	81.69	8014878D0	16m
8004A0940	1.47M	8017792C0	21m

All three transfers are labelled normal, so this is a flag for review — not a fraud finding. Identical \$176.74 amounts cycling through hub 100428660 across three banks in minutes is a lead the graph surfaces to investigate — not a verdict.

Layering & Peeling Through Shared Hubs

FLAG 2 · Internal capital fragmentation. Money enters an intermediary node and leaves immediately at a different value — so amounts never match. A relational JOIN cannot link the two legs; the graph follows the shared account and exposes the pass-through.

PSEUDOCODE

```
SELECT  Origin, Intermediary,
        Dest, $in, $out
WHERE {
  Tx1 senderAccount  Origin ;
      receiverAccount Inter ;
      hasAmountPaid $in ;
      isLaundering false .
  Tx2 senderAccount  Inter ;
      receiverAccount Dest ;
      hasAmountPaid $out ;
      isLaundering false .
  FILTER(time2 > time1)
  FILTER(Origin != Dest)
  FILTER(Inter != Origin, Dest)
}
ORDER BY DESC($in)
```

GRAPHDB RESULT

INTERMEDIARY	IN \$	OUT \$
8017792C0	1.47M	72,108
80EA7F1E0	279,902	556.60
804D57690	216,885	891,268
80407AC60	28,339	152.95
805F9CA10	21,700	2,274
80A431840	10,854	379,047

Rows 1 & 3 chop a lump sum into new values; rows 2 & 4 peel a slice off a large deposit — in/out never match, so a relational JOIN misses it. All are labelled normal, so we flag them as suspicious pass-throughs to check, not proven fraud.

When a Zero Result Is the Finding

We ran two more structural probes over the normal set. Both returned zero rows — not a failure, but evidence. The graph is capable of finding these shapes; their absence shows the original fraud labelling was thorough and well separated.

MULTI-FEEDER ASSET POOL

```
SELECT pool,
       COUNT(feeder) AS feeders
WHERE {
  txIn receiverAccount pool ;
  senderAccount feeder ;
  isLaundering false .
}
GROUP BY pool
HAVING (COUNT(feeder) >= 5)
```

Many senders → one pooling account

0 ROWS — no build-up in normal data

CIRCULAR FLOW LOOP

```
SELECT A, B, C
WHERE {
  A -> B (normal) ;
  B -> C (normal) ;
  C -> A (normal) .
  all hops isLaundering false
  FILTER(A != B != C)
}
# funds return to origin
```

A → B → C → back to A

0 ROWS — no closed loops in normal data

A null result is still a result. Both shapes are structurally detectable, so zero matches means the investigators who labelled this dataset already isolated every feeder-pool and circular-loop case before release — the clean/fraud split holds up under graph scrutiny.

Ensuring the Knowledge Graph is Semantically Sound

Level 1: RDF Structure

10,177 transactions successfully represented in RDF

Every transaction has sender and receiver

All entities have URIs

No orphaned references

Level 2: OWL Semantics

Domain/range enforced

Classes properly disjoint

No contradictions

HermiT confirms:
Zero inconsistencies

Level 3: Query Validation

All queries valid SPARQL

Results trace to RDF data

Relationships correctly followed

100% retrieval accuracy

Semantic correctness: RDF triples follow OWL rules, SPARQL understands relationships, entire graph is logically consistent and auditable.

Ontology Correctness · Graph Completeness · Query Validation · Semantic Reasoning

EV-1: Ontology Correctness

HermiT status: **STABLE**
 Inconsistencies: 0 ✓
 Spurious inferences: 0 ✓
 Disjoint violations: 0 ✓
 Domain/range violations: 0 ✓
 Classes validated: 8 of 8 ✓
 Object properties: 8 of 8 ✓
 Data properties: 4 of 4 ✓

EV-2: Graph Completeness

Transactions loaded: 10,177 ✓
 Fraud cases (true): 5,177 ✓
 Normal cases (false): 5,000 ✓
 Prototype instances: 12 ✓
 Missing values: 0 ✓
 COUNT integrity check: all verified ✓
 CSV→RDF data loss: none ✓

EV-3: Query Competency

Q1 Full network: 12 txns ✓
 Q2 FanOut: Acct_800737690 ✓
 Q3 Cycle: 3-hop loop ✓
 Q4 GatherScatter: Acct_80044A5D0 ✓
 Q5 Leaderboard: 243 hits ✓
 Q6 Pattern totals: \$98K / \$186B ✓
Verified vs IBM ground-truth labels ✓

EV-4: Semantic Reasoning — FraudulentTransaction Inference (NEW)

Defined class: $\text{FraudulentTransaction} \equiv \text{Transaction} \sqcap (\text{belongsToPattern some LaunderingPattern})$. **Result:** HermiT automatically inferred all 12 prototype transactions as members — zero manual assignment (yellow in Protégé). *This proves the ontology reasons over data, deriving new knowledge rather than merely storing it.*

Constraints and Scope Boundaries

Every academic project has boundaries. These are ours — what this prototype does well, where it stops, and what a real-world version would need to go further.

L1

Synthetic Dataset

The data used is artificially created and pre-labelled by IBM. In the real world, fraudulent transactions are not labelled — they have to be identified without knowing in advance which ones are suspicious.

L2

No Real Compliance Integration

This project is a research prototype. It has not been connected to any real banking system or legal compliance framework. The patterns it identifies are for academic demonstration — not for use in actual fraud investigations.

L3

Infrastructure Constraint

The prototype loaded 10,177 transactions. Scaling to the full 5 million is feasible ontologically but computationally demanding — it requires infrastructure beyond a prototype environment. This is a practical constraint of the current setup, not a flaw in the semantic model.

What the Knowledge Graph Proved

Objective 1 Achieved



SPARQL queries traversed the semantic graph and surfaced FanOut, Cycle, and GatherScatter structures invisible in flat data. In the prototype (12 transactions), Account_800737690 emerged as the primary suspect exhibiting both FanOut and Cycle patterns. At scale (5,177 transactions), Account_100428660 dominated as a pure FanOut dispersal hub with 243 confirmed fraud transactions — both identified by network structure alone, no manual labelling required.

Objective 2 Achieved



The ontology can discover suspicious activity in unlabelled data. Testing on transactions marked as normal, the graph surfaced high-velocity accounts, billion-dollar transfers, and volume-concentrated movers — candidates for investigation without pre-labelled fraud tags, proving real-world applicability.

Objective 3 Achieved



HermiT reasoner confirmed zero ontological inconsistencies. The graph is semantically consistent, and every query result traces back to named RDF triples, making the system transparent and auditable.

The knowledge graph provides transparent, explainable, and traceable relationship analysis — without relying on black box machine models

Where This Could Go Next

1. Additional banking entities

Add customers, merchants, locations and device data for a richer picture of each transaction.

4. Larger-scale Infrastructure evaluation

Test the ontology on bigger datasets using more powerful servers and computers to see if the method still works fast and reliably at real-world scale.

2. Expanding fraud indicators and Ontology Schema

Extend the ontology with more entity types and constraints to capture richer fraud signals including unusual frequency, abnormal amounts, high-risk jurisdictions, and additional banking operations. A more detailed schema enables more sophisticated pattern detection.

3. Automated pattern discovery

Let the graph surface unknown structures using graph reasoning or graph machine learning, instead of relying only on hand-written SPARQL.

5. Real-time graph updating

Update the graph as new transactions arrive, rather than from a static historical snapshot.

16 — REFERENCES

What We Took From Each Source

Bennett, M. (2013)

The Financial Industry Business Ontology. **What we applied:** FIBO used as a modelling reference, so our classes align with real financial terminology.

Cantador, I., et al. (2023)

A semantic model for open banking services. **What we applied:** Shaped how we modelled transactions as linked entities rather than flat records.

Liu, Y., et al. (2021)

Graph attention networks for fraud detection. **What we applied:** Showed graph approaches beat flat methods at finding fraud, which justified our whole direction.

Mohsin, S.F., et al. (2024)

Knowledge graphs from financial reports. **What we applied:** Demonstrated KGs scale in finance, supporting that our prototype could grow to real data.

Responsible AI Usage

This project was independently designed, implemented, and analysed by me. Generative AI tools were used in a supplementary capacity only.

Concept Clarification

AI assisted in clarifying concepts related to knowledge graphs, RDF, OWL, and SPARQL during development.

Debugging & Optimisation

AI supported debugging, refinement, and optimisation of Python scripts for data preparation and RDF conversion.

Technical Explanation

AI suggested improvements to the wording, structure, and presentation of technical explanations.

Proofreading & Refinement

AI assisted with proofreading, grammar correction, and slide content refinement.

All ontology design decisions, data preparation choices, SPARQL query logic, GraphDB implementation, evaluation procedures, and conclusions were reviewed, validated, and fully understood by me.